

BT5240 - Computational Systems Biology

K. Varunkumar (BE23B016)

Question 1

a) The yeast protein–protein interaction network was loaded into NetworkX (Undirected Graph). The output of the code to determine network statistics is given below:

```
Number of nodes: 1458
Number of edges: 1948
The density of the above graph is: 0.0018340107310340413
The diameter of the above graph is: 19
```

NOTE: While loading the .mtx file, the node indices were being reset to 0-based indexing, the nodes have been relabelled to 1-based indexing as it is in the dataset.

b) Typically protein-protein interaction networks (PPIN) are used to study the “essentiality” of a protein in a biological system. The three centrality measures we have calculated for the given network are Degree centrality, Betweenness centrality, and Closeness centrality.

- Degree centrality: Determines the influential a protein is (based on the number of interactions it has)
- Betweenness centrality: Determines how often a protein is located on the shortest path between other proteins. A high betweenness indicates a proteins presence in multiple pathways, or its role as a shuttle or a transporter (ex: chaperonins).
- Closeness centrality: Determines the proximity of a protein to other proteins in the network. A high closeness centrality here indicates that the protein could potentially be a master regulator or occupies an upstream position in signalling cascades.

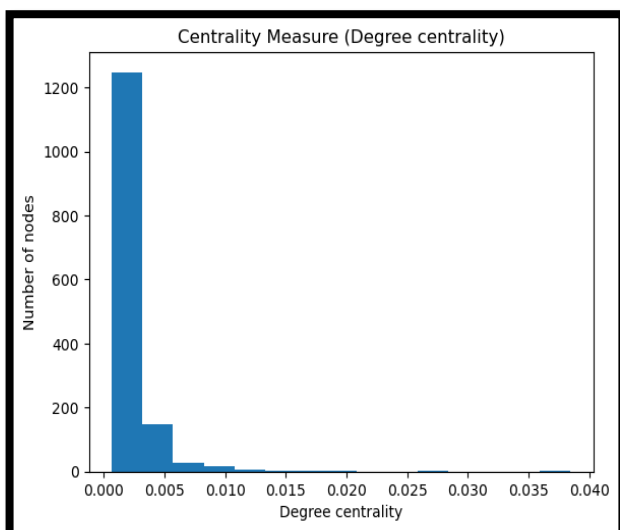


Figure 1.b.1

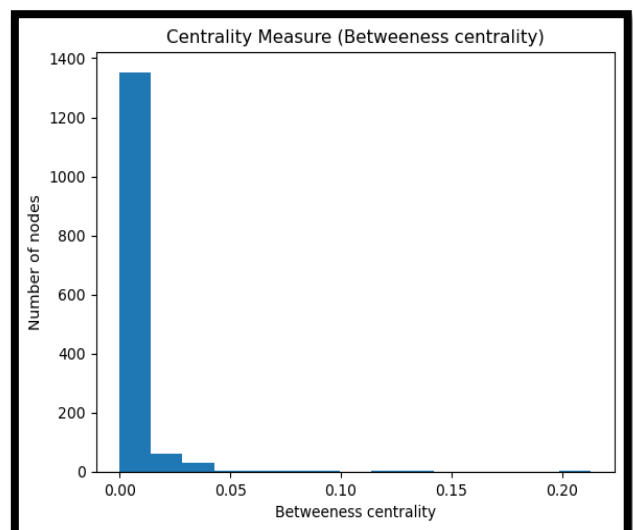


Figure 1.b.2

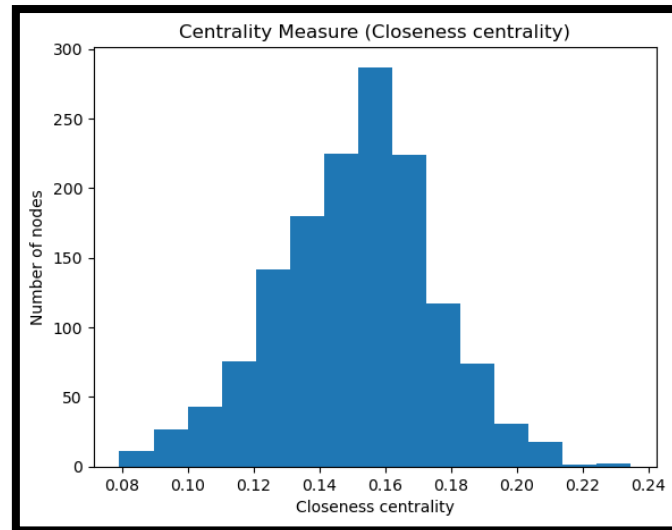


Figure 1.b.3

The above figures illustrate how the distribution of the different centrality measures in this PPIN.

c) The top 5% of nodes based on each centrality measure were computed and the results are given below:

i) The top 5% of nodes with the highest **degree centrality** are shown below:

Top 5% of nodes in the network with the highest degree centrality shown as a list of (degree centrality, node_ID)

```
[(0.038435140700068635, 147), (0.026080988332189428, 819), (0.02059025394646534, 98), (0.01990391214824983, 844), (0.01990391214824983, 567), (0.018531228551818806, 638), (0.017158544955387784, 27), (0.01578586135895676, 1275), (0.014413177762525738, 1240), (0.014413177762525738, 961), (0.013726835964310225, 595), (0.013040494166094714, 367), (0.013040494166094714, 136), (0.012354152367879203, 1130), (0.012354152367879203, 719), (0.012354152367879203, 72), (0.01029512697323267, 433), (0.01029512697323267, 345), (0.009608785175017159, 854), (0.009608785175017159, 257), (0.009608785175017159, 134), (0.008922443376801648, 1315), (0.008922443376801648, 755), (0.008922443376801648, 633), (0.008922443376801648, 355), (0.008922443376801648, 252), (0.008922443376801648, 223), (0.008922443376801648, 122), (0.008236101578586136, 1212), (0.008236101578586136, 979), (0.008236101578586136, 563), (0.008236101578586136, 267), (0.008236101578586136, 154), (0.008236101578586136, 81), (0.0075497597803706245, 1332), (0.0075497597803706245, 1113), (0.0075497597803706245, 1101), (0.0075497597803706245, 920), (0.0075497597803706245, 760), (0.0075497597803706245, 513), (0.0075497597803706245, 301), (0.0075497597803706245, 271), (0.0075497597803706245, 131), (0.0075497597803706245, 25), (0.0075497597803706245, 12), (0.006863417982155113, 1067), (0.006863417982155113, 970), (0.006863417982155113, 942), (0.006863417982155113, 458), (0.006863417982155113, 406), (0.006863417982155113, 281), (0.0061770761839396015, 1368), (0.0061770761839396015, 1041), (0.0061770761839396015, 811), (0.0061770761839396015, 603), (0.0061770761839396015, 524), (0.0061770761839396015, 465), (0.0061770761839396015, 341), (0.0061770761839396015, 333), (0.0061770761839396015, 320), (0.0061770761839396015, 85), (0.0061770761839396015, 11), (0.00549073438572409, 1294), (0.00549073438572409, 1164), (0.00549073438572409, 974), (0.00549073438572409, 791), (0.00549073438572409, 448), (0.00549073438572409, 436), (0.00549073438572409, 348), (0.00549073438572409, 241), (0.004804392587508579, 1273), (0.004804392587508579, 1168)]
```

ii) The top 5% of nodes with the highest **betweenness centrality** are shown below:

Top 5% of nodes in the network with the highest betweenness centrality shown as a list of (betweenness centrality, node_ID)

```
[(0.21299432211070649, 819), (0.13978458296207566, 844), (0.1159242180390304, 98), (0.0985276543252591, 147), (0.07847248516101077, 595), (0.07594949597488396, 1275), (0.06959572524518781, 638), (0.06919433965771314, 567), (0.06627325312176569, 1240), (0.06570532488777521, 253), (0.05345552283122644, 27), (0.04716258789558213, 961), (0.042258091178716234, 465), (0.04097842770251197, 90), (0.04079213505611553, 664), (0.040229913041750984, 302), (0.03881344048477032, 1101), (0.03873312947303164, 241), (0.0382197133874563, 122), (0.03635829054418146, 367), (0.03598824631355666, 384), (0.03549083689199403, 979), (0.035417439052021715, 1), (0.03510158701752763, 1130), (0.034436366177166, 337), (0.03397153564796833, 1131), (0.0337834169546319, 136), (0.032950960208735794, 854), (0.03282562701462217, 17), (0.03256249301550081, 920), (0.032442949026390375, 719), (0.03222709848470551, 1164), (0.03213273055203752, 433), (0.031592078180595236, 1168), (0.03134321496081657, 72), (0.031190907738765265, 355), (0.03099307615665204, 266), (0.03042892334783725, 563), (0.02997254838780976, 257), (0.029962079246366315, 267), (0.02993845445661677, 1294), (0.029782828442136134, 585), (0.029080606899538765, 445), (0.027734115719775555, 1212), (0.02723832175354469, 603), (0.026942956981392707, 535), (0.026733417201486066, 584), (0.026285175205262114, 345), (0.02611982261656477, 1392), (0.02536035275447479, 689), (0.025080170227142106, 252), (0.02479472718499414, 281), (0.024705190899214094, 1110), (0.024520877846849583, 464), (0.022889736442611643, 1014), (0.02235699460473996, 942), (0.021928423385857414, 1132), (0.021601728596062634, 1041), (0.02150872105591344, 154), (0.02122919913402316, 1113), (0.021106825023498567, 458), (0.020985281733532608, 1368), (0.020903603878953746, 1273), (0.020852803516322935, 970), (0.02069697613572957, 572), (0.02063310928872657, 998), (0.020381297113369983, 802), (0.02028831829005771, 204), (0.020239013334025304, 966), (0.02014267563820602, 322), (0.020054876743817757, 513), (0.020039995762221038, 11)]
```

iii) The top 5% of nodes with the highest **closeness centrality** are shown below:

```
Top 5% of nodes in the network with the highest closeness centrality shown as a list of (closeness centrality, node_ID)
[(0.23477280051563004, 819), (0.22941269091481656, 844), (0.21903187011425135, 98), (0.21205064764954154, 17), (0.2091888011486
0014, 302), (0.20861970217640322, 90), (0.20847045356989555, 179), (0.20808340474150241, 966), (0.2080536912751678, 1131), (0.2
076385919908793, 567), (0.2075794272688417, 731), (0.20637393767705384, 355), (0.20564573041637263, 433), (0.2052690898844745,
664), (0.20480742198481866, 1240), (0.20460609464962787, 253), (0.2042905215928211, 252), (0.20403304859263408, 609), (0.203947
36842105263, 513), (0.20374772759054677, 585), (0.20371923937360178, 241), (0.20295305752890375, 13), (0.20264255910987483, 33
7), (0.20194040194040194, 204), (0.20174466906674052, 551), (0.20166089965397924, 1172), (0.20146570796460178, 464), (0.2013543
394140409, 1171), (0.20091009376723662, 595), (0.20082701585113716, 465), (0.20044022561562802, 937), (0.19947973713033954, 58
4), (0.1988264192139738, 1266), (0.19844728956687552, 180), (0.19836623553437713, 1273), (0.19777385638658884, 1077), (0.197505
7611495188, 638), (0.19723839176932448, 324), (0.19694512030278455, 979), (0.19689189189189188, 445), (0.1967323791520389, 60
3), (0.19657312466270913, 854), (0.19657312466270913, 122), (0.19612330057881275, 1275), (0.19601775864388538, 563), (0.1958070
151861309, 1132), (0.19551798174986582, 384), (0.1953344952406489, 1130), (0.1943962641761174, 261), (0.19380154296355415, 136
8), (0.19375, 267), (0.19359553547701303, 662), (0.1931592204693093, 681), (0.19310801855533466, 920), (0.19262295081967212, 11
15), (0.19254658385093168, 152), (0.1921403138599499, 572), (0.19191253951527923, 791), (0.19173575470456639, 494), (0.19171052
631578947, 27), (0.1916600894501447, 85), (0.19163488096803893, 8), (0.19138316038355443, 332), (0.1912826572141263, 579), (0.1
9120734908136483, 604), (0.190956749672346, 1035), (0.1908316961362148, 1212), (0.1907567426027756, 524), (0.19063195080465786,
184), (0.19060701203558347, 435), (0.1904077365394668, 831), (0.1903579827541155, 526)]
```

d) Each set of putative nodes was removed independently from the network and the size of the largest connected component was computed as shown below:

```
The size of the largest connected component before removal of any nodes is: 1458
The size of the largest connected component after removal of putative proteins (degree centrality based) is: 613
The size of the largest connected component after removal of putative proteins (betweenness centrality based) is: 681
The size of the largest connected component after removal of putative proteins (closeness centrality based) is: 1117
```

NOTE: At the cutoff for putative nodes, multiple nodes may share identical centrality values. As such the exact size of the connected component might change based on the nodes included into the set of putative proteins in case of tie in centrality value.

e) Shown below is the distribution of largest connected component obtained upon 100 iterations of random removal of 5% of nodes from the PPIN.

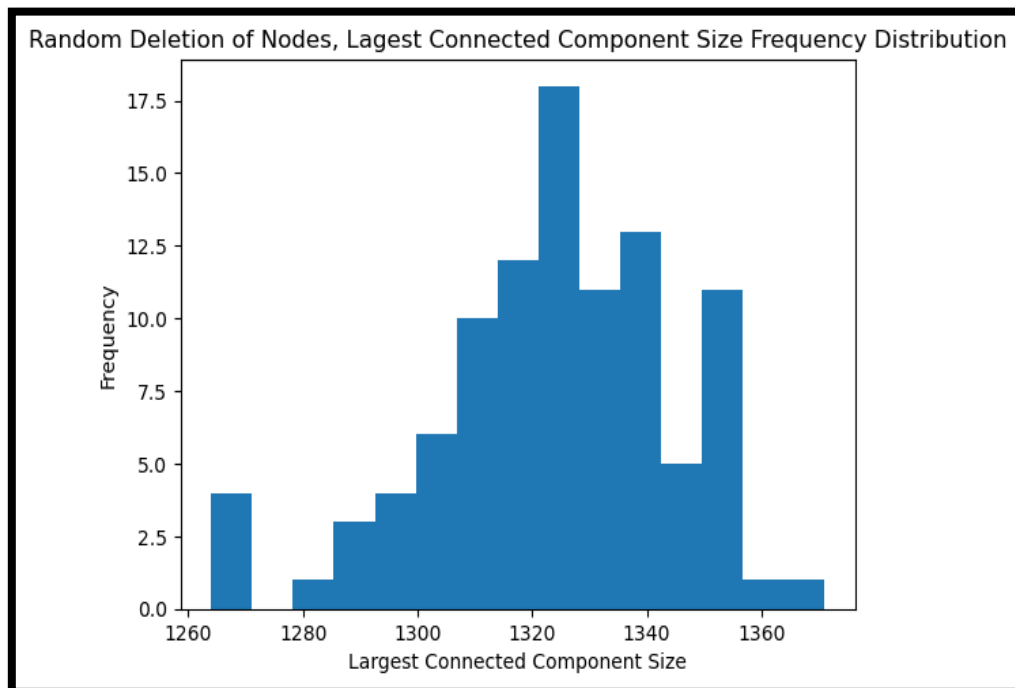


Figure 1.e.1

The mean and variance of the distribution was calculated. The Z-score was computed using the formula:

$$Z = \frac{x - \mu}{\sigma}$$

- “x” denotes the observed value
- “ μ ” denotes the mean of the distribution
- “ σ ” denotes the standard deviation of the distribution

H_0 : There is no statistically significant difference in the size of the largest connected component between the removal of nodes with the highest degree centrality and random removal of nodes in this network.

H_1 : There is a statistically significant difference in the size of the largest connected component between the removal of nodes with the highest degree centrality and random removal of nodes in this network.

The output for the two tailed Z-tests are given below:

i) Removal of nodes with highest degree centrality:

Calculated Z score = -33.809
 Critical Z value = 1.96
 Reject the null hypothesis ($p = 1.4322920734654068e-250$)

ii) Removal of nodes with highest betweenness centrality:

Calculated Z score = -30.575
 Critical Z value = 1.96
 Reject the null hypothesis ($p = 2.6381328017868483e-205$)

iii) Removal of nodes with highest closeness centrality:

Calculated Z score = -9.836
 Critical Z value = 1.96
 Reject the null hypothesis ($p = 7.890572493397286e-23$)

In terms of the centrality-lethality hypothesis (which posits that nodes central in a biological network are far more likely to be essential for cellular function compared to nodes with lesser centrality) the choice of centrality measures is as follows: **Degree centrality > Betweenness centrality > Closeness centrality.**

NOTE: In this case it is not very appropriate to carry out a Z-test as we don't actually know the population standard deviation (σ), as such it is far more appropriate to carry out a student's t-test (with d.o.f = 99 in this case) and use sample variance (s_x).

f)

Nodes 1, 17, 266, 535, 1392 are the top 5 nodes in terms of betweenness centrality with degree ≤ 5

$[(0.035417439052021715, 1), (0.03282562701462217, 17), (0.03099307615665204, 266), (0.026942956981392707, 535), (0.02611982261656477, 1392)]$

Nodes 266, 535, 1014, 322, 1242 are the top 5 nodes in terms of betweenness centrality with degree ≤ 3

```
[(0.03099307615665204, 266), (0.026942956981392707, 535), (0.022889736442611643, 1014), (0.02014267563820602, 322), (0.020016291348488757, 1242)]
```

g) The top 5 nodes with the highest closeness centrality in the yeast PPIN are:

```
The protein YDL160C has the highest closeness centrality = 0.6851443569553806  
The protein YAL021C has the second highest closeness centrality = 0.6469072164948454  
The protein YGL122C has the third highest closeness centrality = 0.6214646224169127  
The protein YFL039C has the forth highest closeness centrality = 0.5928415697674418  
The protein YNL209W has the fifth highest closeness centrality = 0.5856591582159203
```

Question 2

1) The figure below shows Zachary's Karate Club Network, it has 2 communities based on node label which are Mr. Hi and Officer.

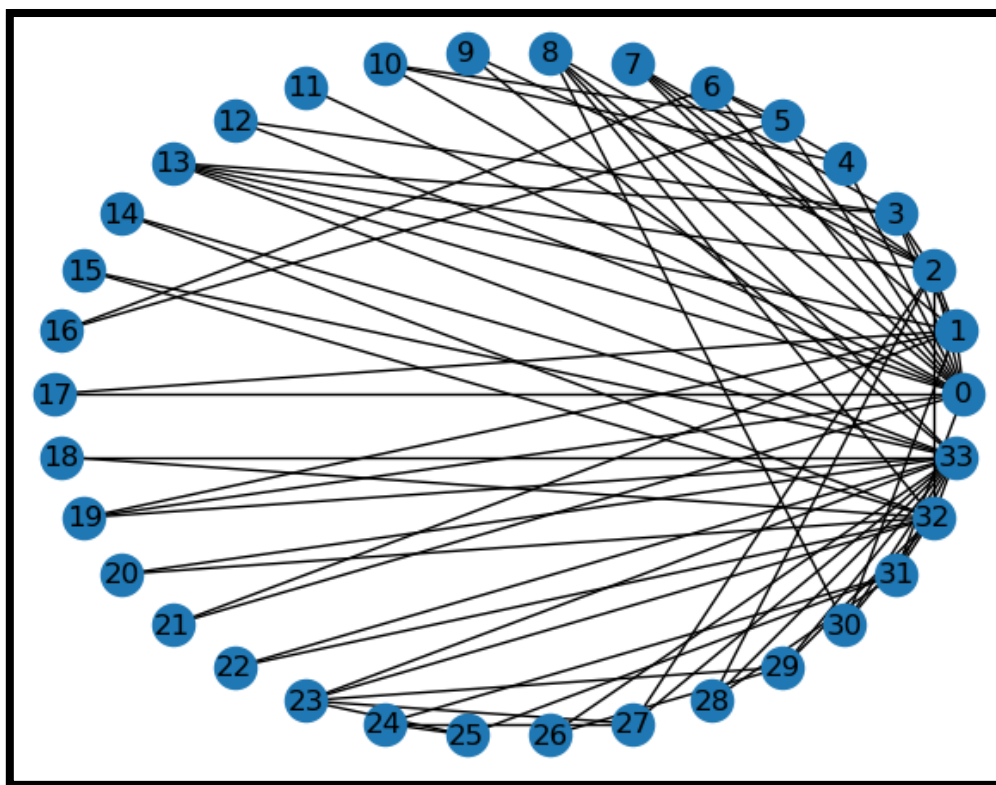


Figure 2.1.1

The true communities are given below:

```
The true communities as per labels are:  
Mr.Hi community: {0, 1, 2, 3, 4, 5, 6, 7, 8, 10, 11, 12, 13, 16, 17, 19, 21}  
Officer community: {32, 33, 9, 14, 15, 18, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31}
```

Greedy Modularity Maximization while restricting the maximum number of communities to 2, 3, and 4 was executed and the output is shown below:

```
For part (a), (i), maximum number of communities = 2
Community 1: {0, 1, 2, 3, 4, 5, 6, 7, 9, 10, 11, 12, 13, 16, 17, 19, 21}, it's size is: 17
Community 2: {8, 14, 15, 18, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33}, it's size is: 17
The modularity value is: 0.3998332115215232
=====
For part (a), (ii), maximum number of communities = 3
Community 1: {8, 14, 15, 18, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33}, it's size is: 17
Community 2: {1, 2, 3, 7, 9, 12, 13, 17, 21}, it's size is: 9
Community 3: {0, 4, 5, 6, 10, 11, 16, 19}, it's size is: 8
The modularity value is: 0.41096493693896297
=====
For part (a), (iii), maximum number of communities = 4
Community 1: {8, 14, 15, 18, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33}, it's size is: 17
Community 2: {1, 2, 3, 7, 9, 12, 13, 17, 21}, it's size is: 9
Community 3: {0, 4, 5, 6, 10, 11, 16, 19}, it's size is: 8
The modularity value is: 0.41096493693896297
```

Modularity increases as number of communities increases from 2 to 3. However, the algorithm doesn't classify more than 3 communities even when it is allowed a maximum of 4 communities is allowed. When the number communities is forced to being 4, we see below that the modularity actually decreases.

```
For part (a), (i), number of communities = 2
Community 1: {0, 1, 2, 3, 4, 5, 6, 7, 9, 10, 11, 12, 13, 16, 17, 19, 21}, it's size is: 17
Community 2: {8, 14, 15, 18, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33}, it's size is: 17
The modularity value is: 0.3998332115215232
=====
For part (a), (ii), number of communities = 3
Community 1: {8, 14, 15, 18, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33}, it's size is: 17
Community 2: {1, 2, 3, 7, 9, 12, 13, 17, 21}, it's size is: 9
Community 3: {0, 4, 5, 6, 10, 11, 16, 19}, it's size is: 8
The modularity value is: 0.41096493693896297
=====
For part (a), (iii), number of communities = 4
Community 1: {8, 14, 15, 18, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33}, it's size is: 17
Community 2: {1, 2, 3, 7, 9, 12, 13, 17, 21}, it's size is: 9
Community 3: {0, 16, 4, 5, 6, 10, 11}, it's size is: 7
Community 4: {19}, it's size is: 1
The modularity value is: 0.40671089372388075
```

In terms of accurately capturing the true structure, the split with the highest modularity produces 3 communities. However, this does not even remotely resemble the real structure of the network, since the network is known to consist of only 2 communities.

2) The Girvan-Newman algorithm was run, the first five successive partitions produced are shown below:

```
Partition number 1
Number of communities: 2
Community 1: {0, 1, 3, 4, 5, 6, 7, 10, 11, 12, 13, 16, 17, 19, 21}
Community 2: {2, 8, 9, 14, 15, 18, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33}
The sizes of the community are: [15, 19]
The modularity is: 0.3476602762317048
=====
Partition number 2
Number of communities: 3
Community 1: {0, 1, 3, 4, 5, 6, 7, 10, 11, 12, 13, 16, 17, 19, 21}
Community 2: {2, 8, 14, 15, 18, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33}
Community 3: {9}
The sizes of the community are: [1, 15, 18]
The modularity is: 0.3423192968647514
=====
Partition number 3
Number of communities: 4
Community 1: {0, 1, 3, 7, 11, 12, 13, 17, 19, 21}
Community 2: {2, 8, 14, 15, 18, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33}
Community 3: {16, 4, 5, 6, 10}
Community 4: {9}
The sizes of the community are: [1, 5, 10, 18]
The modularity is: 0.3580611307884035
=====
```

```

Partition number 4
Number of communities: 5
Community 1: {0, 1, 3, 7, 11, 12, 13, 17, 19, 21}
Community 2: {2, 24, 25, 27, 28, 31}
Community 3: {16, 4, 5, 6, 10}
Community 4: {32, 33, 8, 14, 15, 18, 20, 22, 23, 26, 29, 30}
Community 5: {9}
The sizes of the community are: [1, 5, 6, 10, 12]
The modularity is: 0.3849721706864564
=====

```

```

Partition number 5
Number of communities: 6
Community 1: {0, 1, 3, 7, 12, 13, 17, 19, 21}
Community 2: {2, 24, 25, 27, 28, 31}
Community 3: {16, 4, 5, 6, 10}
Community 4: {32, 33, 8, 14, 15, 18, 20, 22, 23, 26, 29, 30}
Community 5: {9}
Community 6: {11}
The sizes of the community are: [1, 1, 5, 6, 9, 12]
The modularity is: 0.37578006409175235
=====

```

The partition with maximum modularity is obtained upon the removal of the 4th edge as shown above, producing 5 communities.

As the edges are progressively removed the modularity initially increase (from 1st to 4th edge removal), however it decreases upon the removal of the 5th edge.

In terms of accurately capturing the true structure, the split with the highest modularity produces 5 communities, which is worse even when compared to the Greedy Modularity Maximization algorithm which predicted 3 communities. Thus, the predicted communities here as well does not even remotely resemble the real structure of the network.

3) The maximum modularity obtained using both algorithms is given below:

```

The true modularity is: 0.39143756676224206
=====
The max modularity in Girvan-Newman is: 0.3849721706864564
The max modularity using the Greedy modularity method is: 0.41096493693896297
=====

```

Eventhough the Girvan-Newman algorithm has 5 communities in the partition with maximum modularity it still has a lower overall maximum modularity when compared to the Greedy Modularity Maximization algorithm, whose max modularity set had only 3 communities.

Analysing the time complexity of both Algorithms:

i) Greedy Modularity Algorithm:

- Start of with each node as an individual community.
- Make all possible set of comparisons and determine the merger of which pair of communities increases the modularity the most
- Join those 2 communities to form a new (larger) community and treat their group as a single entity.
- Now again performs steps 2 and 3 repeatedly until no new merger that increases the modularity is possible.

Thus, if we were to assume the time to compute modularity is $O(1)$, we can estimate the complexity of the Naïve Greedy Modularity Algorithm as shown below:

$$\sum_{q=1}^n \frac{q \cdot (q - 1)}{2}$$

Every time communities merge there are q to $q-1$ communities, which is how we obtained this sum.

Using this we can now clearly see the Greedy Modularity Maximization Algorithm is $O(n^3)$.

The source code is given in:

https://networkx.org/documentation/stable/_modules/networkx/algorithms/community/modularity_max.html#greedy_modularity_communities

The current implementation of the Greedy Modularity Maximization Algorithm however doesn't follow the naïve algorithm given above, it instead uses the method given in “**Finding community structure in very large networks**”, this algorithm is a significant improvement on the naïve algorithm and uses a dendrogram for this purpose.

The improved algorithm according to the authors has a complexity of $O(md \log n)$, where m and n have the same meaning as before and ‘ d ’ denotes the depth of the dendrogram.

The discovery and analysis of community structure in networks is a topic of considerable recent interest within the physics community, but most methods proposed so far are unsuitable for very large networks because of their computational cost. Here we present a hierarchical agglomeration algorithm for detecting community structure which is faster than many competing algorithms: its running time on a network with n vertices and m edges is $O(md \log n)$ where d is the depth of the dendrogram describing the community structure. Many real-world networks are sparse and hierarchical, with $m \sim n$ and $d \sim \log n$, in which case our algorithm runs in essentially linear time, $O(n \log^2 n)$. As an example of the application of this algorithm we use it to analyze a network of items for sale on the web site of a large on-line retailer, items in the network being linked if they are frequently purchased by the same buyer. The network has more than 400 000 vertices and 2×10^6 edges. We show that our algorithm can extract meaningful communities from this network, revealing large-scale patterns present in the purchasing habits of customers.

Figure 2.3.1 (Source: <https://journals.aps.org/pre/pdf/10.1103/PhysRevE.70.066111>)

ii) Girvan-Newman Algorithm:

To compute the betweenness centrality Brandes' algorithm is used and it has a complexity of $O(mn)$.

Algorithms [\[edit\]](#)

Calculating the betweenness and [closeness centralities](#) of all the [vertices](#) in a graph involves calculating the shortest paths between all pairs of vertices on a graph, which takes $\Theta(|V|^3)$ time with the [Floyd–Warshall algorithm](#), modified to not only find one but count all shortest paths between two nodes. On a sparse graph, [Johnson's algorithm](#) or [Brandes' algorithm](#) may be more efficient, both taking $O(|V|^2 \log |V| + |V||E|)$ time. On unweighted graphs, calculating betweenness centrality takes $O(|V||E|)$ time using Brandes' algorithm.^[5]

Figure 2.3.2 (Source: https://en.wikipedia.org/wiki/Betweenness_centrality#Definition)

From the implementation of Girvan-Newman discussed in class, we can say that the Girvan-Newman Algorithm has a complexity determined by

$$n \cdot \sum_{r=1}^m \frac{r}{2}$$

The computation for edge betweenness is $O(rn)$, when there is r edges (this is done from $r = 1$ to $r = m$)

Thus Girvan-Newman has a complexity of $O(m^2n)$

The source code is given in:

https://networkx.org/documentation/stable/_modules/networkx/algorithms/community/centrality.html#girvan_newman

Now to compare the accuracy of both methods, we took the partitions sets of both algorithms where they classified 2 communities, and compare their accuracy.

```
The accuracy of the Greedy modularity algorithm in detecting the communities is: 94.11764705882354
The accuracy of the Girvan-Newman algorithm in detecting the communities is: 94.11764705882354
```

Both algorithms misclassify 2 nodes out of the 34 nodes, and end up having the same accuracy.

If the number of communities was known beforehand, we see that both algorithms end up having the same accuracy of classification.

4) Looking at the source code for Girvan-Newman we see that, the node to be removed, is the first occurrence in the dictionary 'betweenness' with the highest value. Therefore, in case of a tie, the node with the highest edge betweenness that is first in the dictionary is removed.

```
if G.number_of_edges() == 0:
    yield tuple(nx.connected_components(G))
    return
# If no function is provided for computing the most valuable edge,
# use the edge betweenness centrality.
if most_valuable_edge is None:

    def most_valuable_edge(G):
        """Returns the edge with the highest betweenness centrality
        in the graph `G`."""

        # We have guaranteed that the graph is non-empty, so this
        # dictionary will never be empty.
        betweenness = nx.edge_betweenness centrality(G)
        return max(betweenness, key=betweenness.get)

# The copy of G here must include the edge weight data.
g = G.copy().to_undirected()
# Self-loops must be removed because their removal has no effect on
# the connected components of the graph.
g.remove_edges_from(nx.selfloop_edges(g))
while g.number_of_edges() > 0:
    yield _without_most_central_edges(g, most_valuable_edge)
```

Figure 2.4.1

https://networkx.org/documentation/stable/_modules/networkx/algorithms/community/centrality.html#girvan_newman (Source for the above image)

Ideally it should be the first edge encountered, that has the highest edge betweenness centrality value, however since a dictionary isn't ordered, we cannot determine edge is output first.

NOTE: This doesn't mean the algorithm is not deterministic, as it gives the same output each time, just that it is not possible to predict the output without running the program in the first place.

5) In question 1 we see that even when we set the maximum allowed communities to 4, the Greedy Modularity Maximization Algorithm still returns only 3 communities, this is a clear case of the algorithm merging small, distinct clusters into a single community to artificially boost the modularity score.

```

For part (a), (i), maximum number of communities = 2
Community 1: {0, 1, 2, 3, 4, 5, 6, 7, 9, 10, 11, 12, 13, 16, 17, 19, 21}, it's size is: 17
Community 2: {8, 14, 15, 18, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33}, it's size is: 17
The modularity value is: 0.3998332115215232
=====

```

```

For part (a), (ii), maximum number of communities = 3
Community 1: {8, 14, 15, 18, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33}, it's size is: 17
Community 2: {1, 2, 3, 7, 9, 12, 13, 17, 21}, it's size is: 9
Community 3: {0, 4, 5, 6, 10, 11, 16, 19}, it's size is: 8
The modularity value is: 0.41096493693896297
=====

```

```

For part (a), (iii), maximum number of communities = 4
Community 1: {8, 14, 15, 18, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33}, it's size is: 17
Community 2: {1, 2, 3, 7, 9, 12, 13, 17, 21}, it's size is: 9
Community 3: {0, 4, 5, 6, 10, 11, 16, 19}, it's size is: 8
The modularity value is: 0.41096493693896297
=====

```

```

For part (a), (i), number of communities = 2
Community 1: {0, 1, 2, 3, 4, 5, 6, 7, 9, 10, 11, 12, 13, 16, 17, 19, 21}, it's size is: 17
Community 2: {8, 14, 15, 18, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33}, it's size is: 17
The modularity value is: 0.3998332115215232
=====

```

```

For part (a), (ii), number of communities = 3
Community 1: {8, 14, 15, 18, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33}, it's size is: 17
Community 2: {1, 2, 3, 7, 9, 12, 13, 17, 21}, it's size is: 9
Community 3: {0, 4, 5, 6, 10, 11, 16, 19}, it's size is: 8
The modularity value is: 0.41096493693896297
=====

```

```

For part (a), (iii), number of communities = 4
Community 1: {8, 14, 15, 18, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33}, it's size is: 17
Community 2: {1, 2, 3, 7, 9, 12, 13, 17, 21}, it's size is: 9
Community 3: {0, 16, 4, 5, 6, 10, 11}, it's size is: 7
Community 4: {19}, it's size is: 1
The modularity value is: 0.40671089372388075

```

As we see from the above when we force the algorithm to generate 4 clusters, the total modularity score indeed reduces.

Question 3

1) To obtain the degree distribution, boot up Cytoscape, remove the unwanted lines from the Excel sheet by opening it using Notepad, then Import the File as Network into Cytoscape.

Then do Tools => Analyze Network => Node Degree distribution (In the analyzer)

Customize the Degree distribution plots binwidth, axis etc.

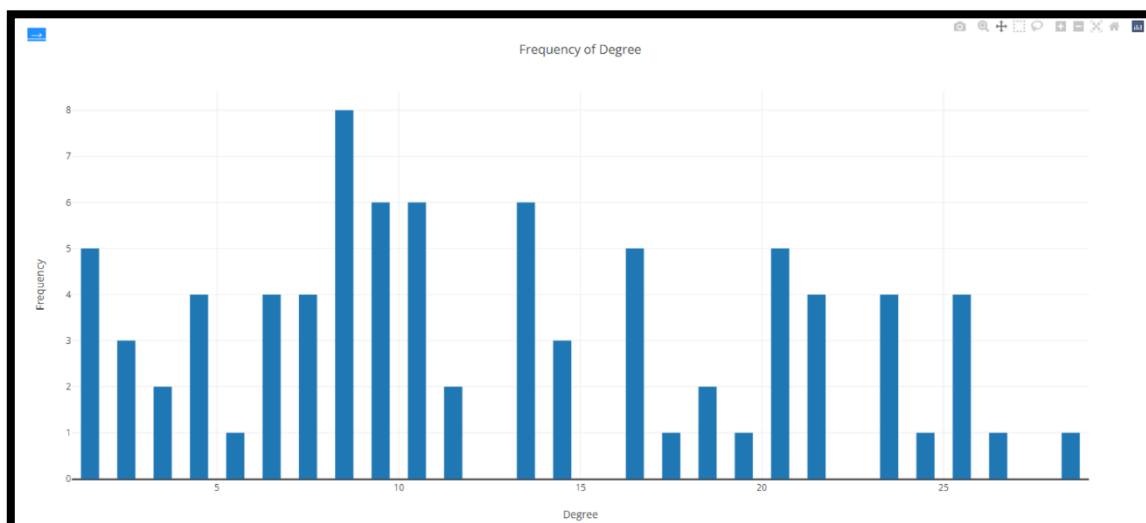


Figure 3.1.1: Hospital Network Degree Distribution

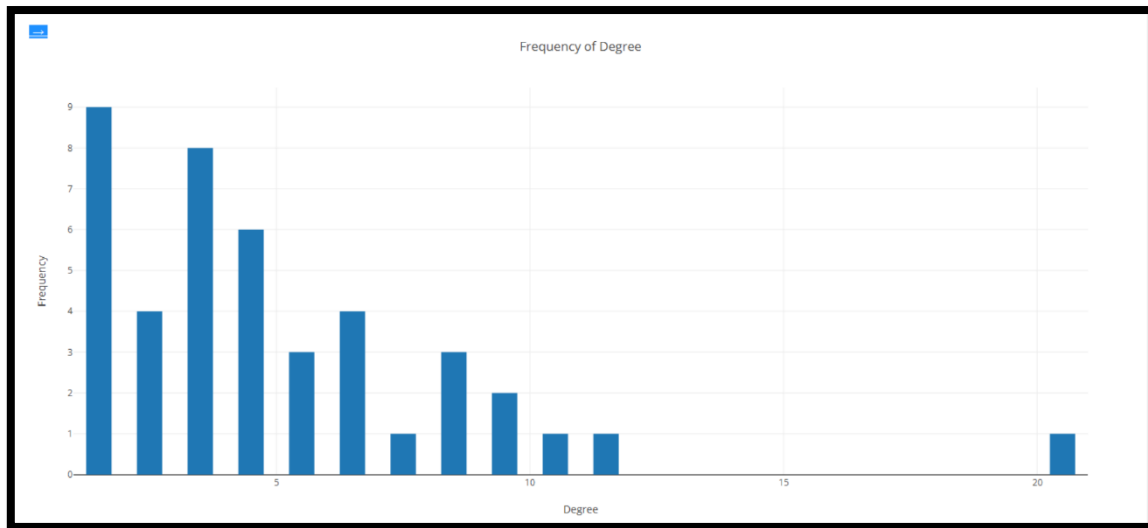


Figure 3.1.2: Metro Network Degree Distribution

From the above Degree-distributions we can clearly see that the Hospital Network is far more inter-connected and dense when compared to the Metro Network.

2) The average clustering coefficient of both networks was calculated using python and the plots are shown below:

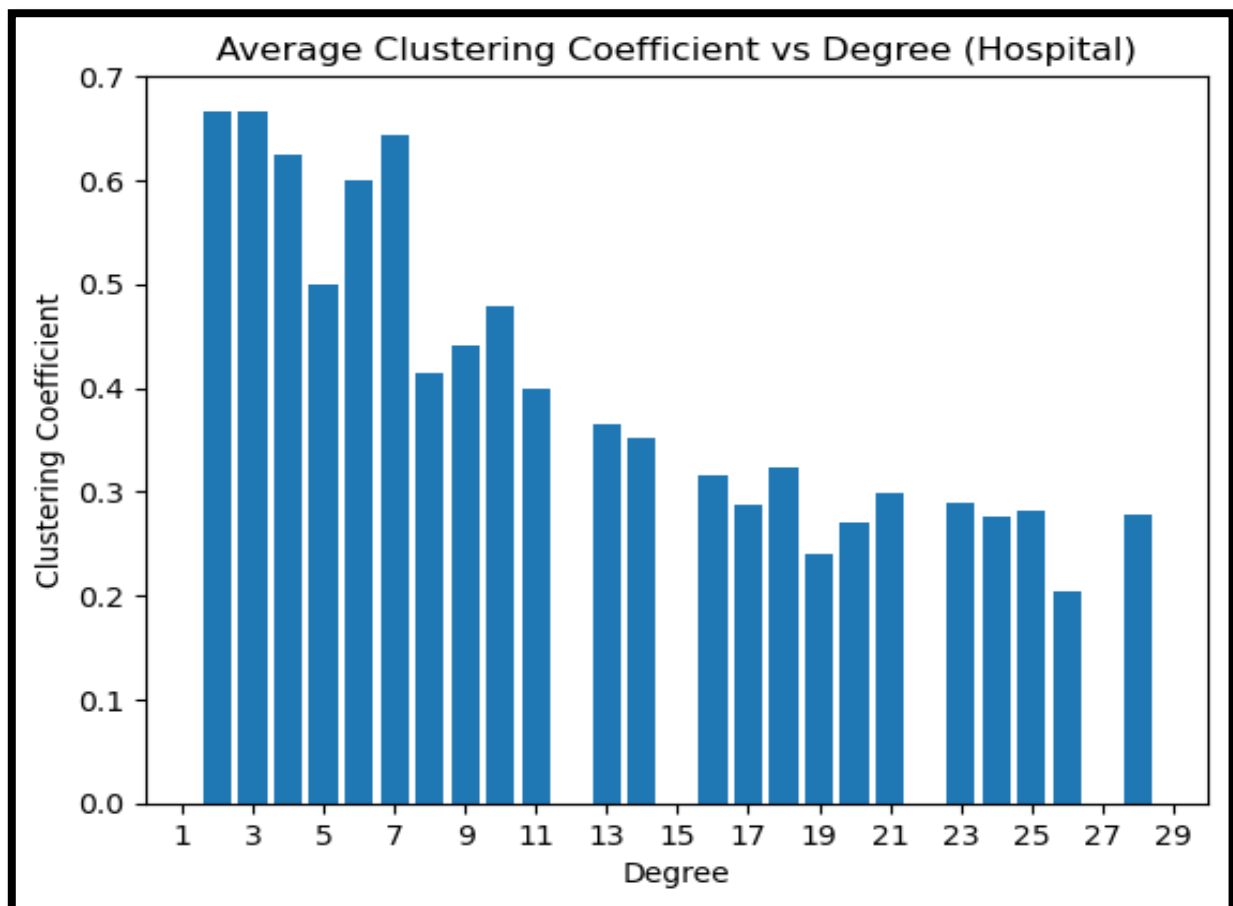


Figure 3.2.1: Average Clustering Coefficient vs Degree (Hospital)

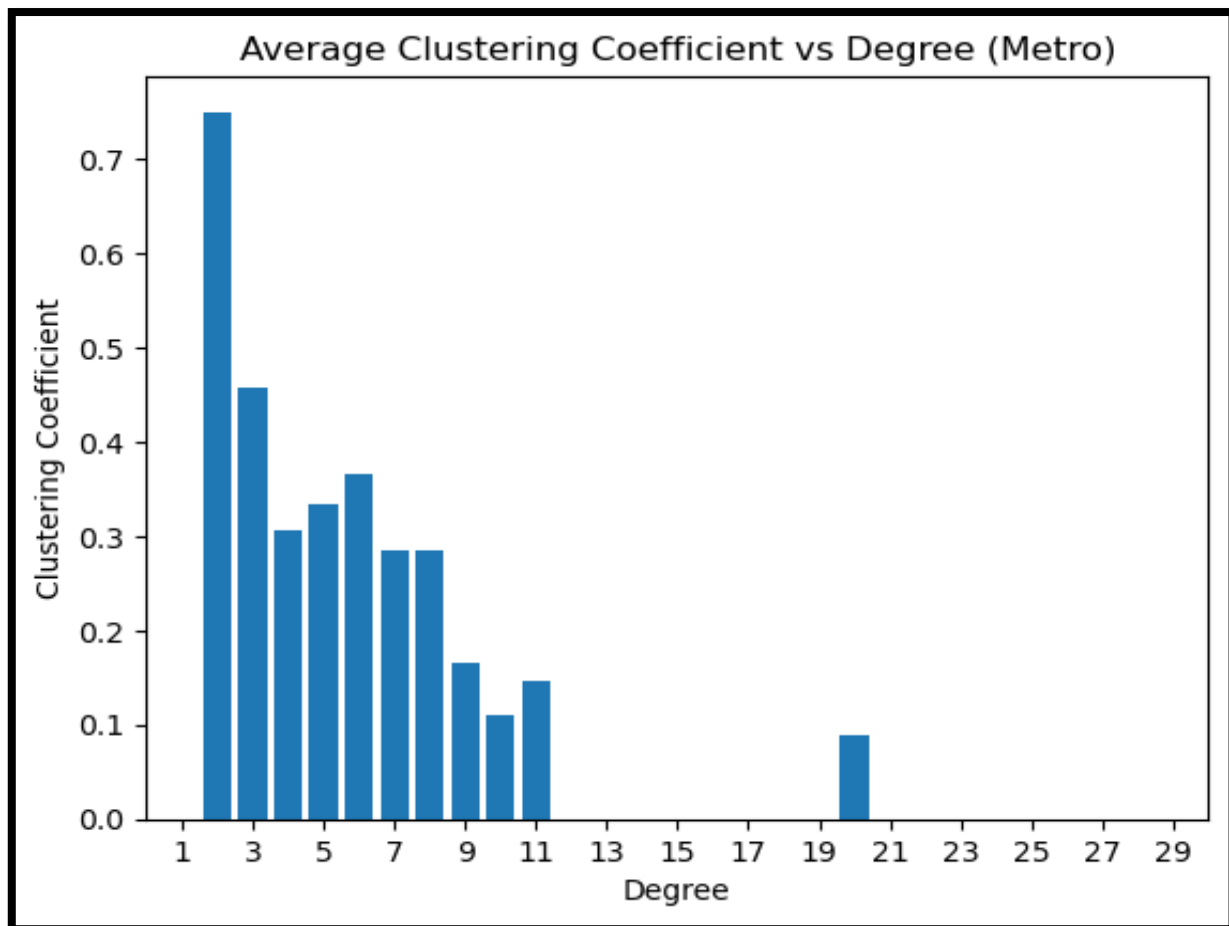


Figure 3.2.2: Average Clustering Coefficient vs Degree (Metro)

From the above graph we can see that there are more nodes in the Hospital Network that have a high clustering coefficient as compared to the Metro Network, indicating that the Hospital Network is more dense and has greater number of microbial associations when compared to the Metro Network.

3) To obtain the required data: Highlight Network => Tools => Analyze Network (Undirected Graph)

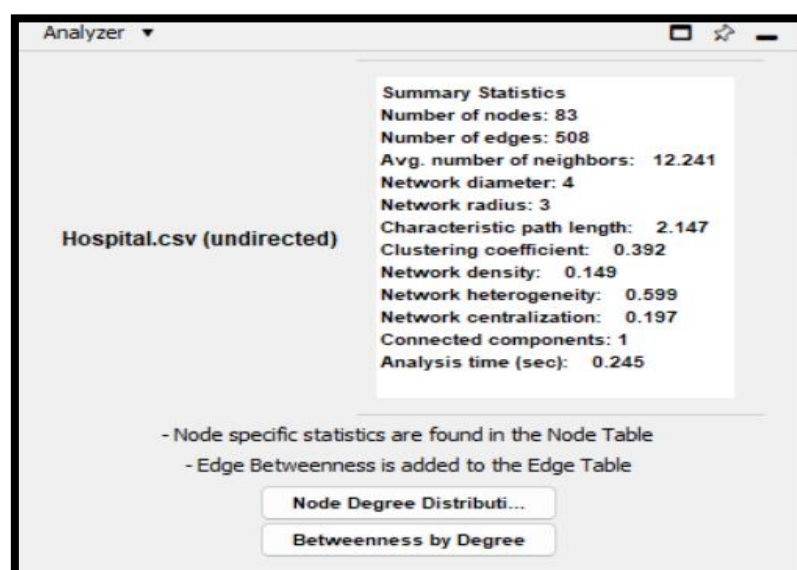


Figure 3.3.1: Statistics of the Hospital Network

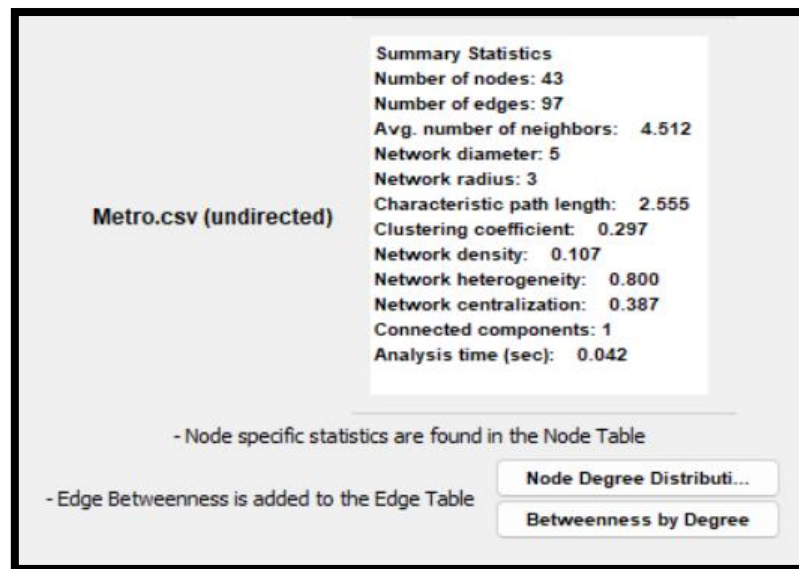


Figure 3.3.2: Statistics of the Metro Network

Python code was executed for the same and the output is given below:

```
The average node degree in the microbial association networks from the Hospital is: 12.240963855421686
The average clustering coefficient in the microbial association networks from the Hospital is: 0.3916766098207763
The average characteristic path length in the microbial association networks from the Hospital is: 2.146929180135175
=====
The average node degree in the microbial association networks from the Metro is: 4.511627906976744
The average clustering coefficient in the microbial association networks from the Metro is: 0.2974161972937983
The average characteristic path length in the microbial association networks from the Metro is: 2.5548172757475083
```

From the above, we are able to see that average degree in the Hospital network is greater (as was expected from part 1), we see that the average clustering coefficient is higher for the Hospital Network (as was expected from part 2), We also happen to see here that the average characteristic path length in the Hospital network is much lower, indicating that as we concluded in part 2, it is indeed much more densely connected.

4) To obtain the common nodes in both networks:

Tools => Merge => Networks => Intersection => Network 1 and 2 are selected and are Merged

To obtain the set difference between the two networks:

Tools => Merge => Networks => Intersection => Network 1,2 are selected => Removal all nodes that are in the 2nd network.

Doing this for both Hospital-Metro and Metro-Hospital gives both the unique interaction networks

The Intersection table data was exported to compare the common microbial interaction strengths, using python the plots given below were produced.

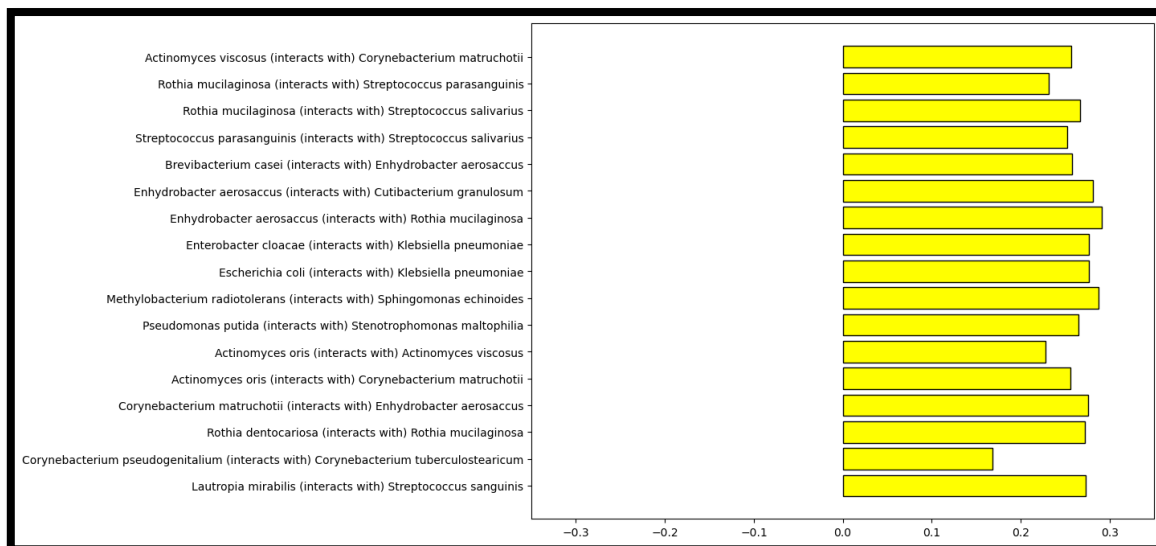


Figure 3.4.1: Common Nodes (Hospital), normalized correlation values

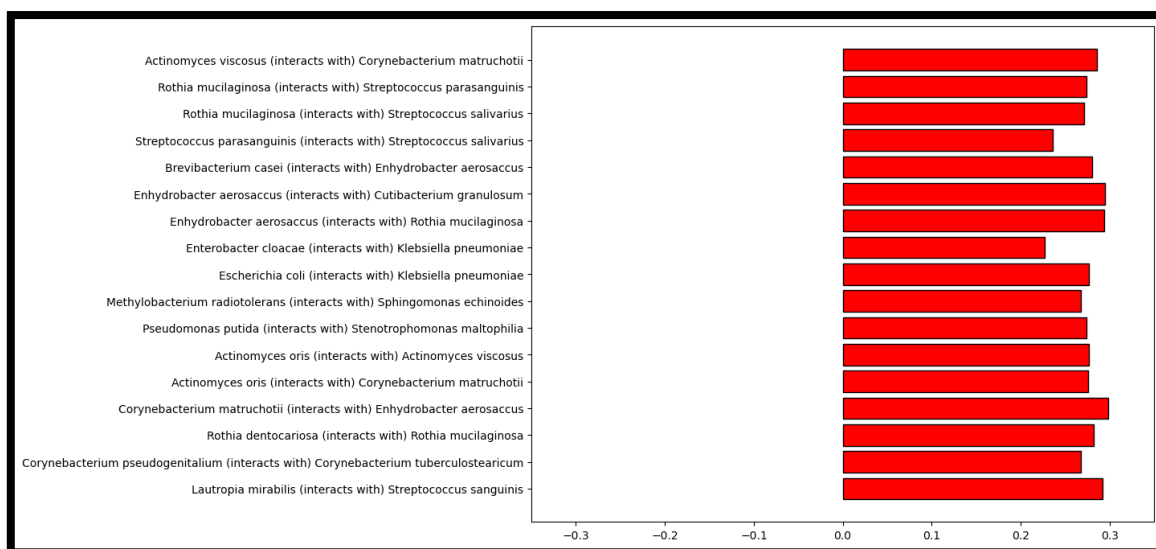


Figure 3.4.2: Common Nodes (Metro), normalized correlation values

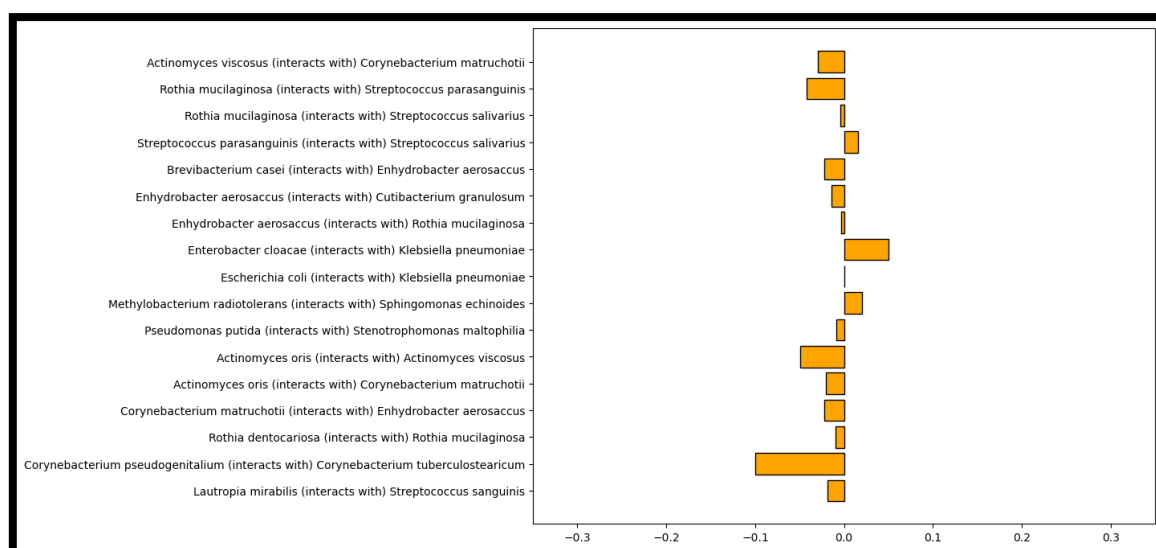


Figure 3.4.3: Common Nodes (Difference; Hospital-Metro), normalized correlation values

From the above plots we see that the microbes are interacting, to varying extents in both environments.

For example, *Corynebacterium pseudogenitalium*'s interaction with *Corynebacterium tuberculo*stearicum shows large deviation in terms of strength of interaction between both networks (with the interaction being much weaker in the Hospital environment).

In general *Enhydrobacter aerosaccus*'s strength of interactions don't seem to vary much between environments.

Apart from a couple of outliers, we see that in general the strength of interactions aren't all that different.

ii) Sorting by degree for identifying microbes with high interactions.

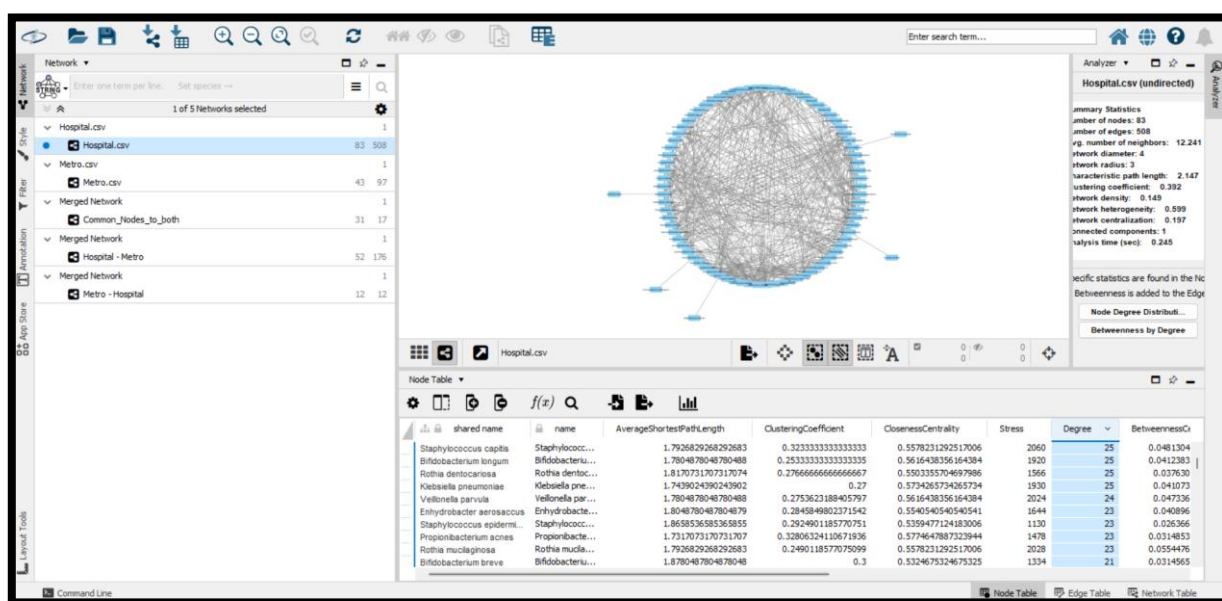


Figure 3.4.4: Top interacting microbes in the Hospital Network

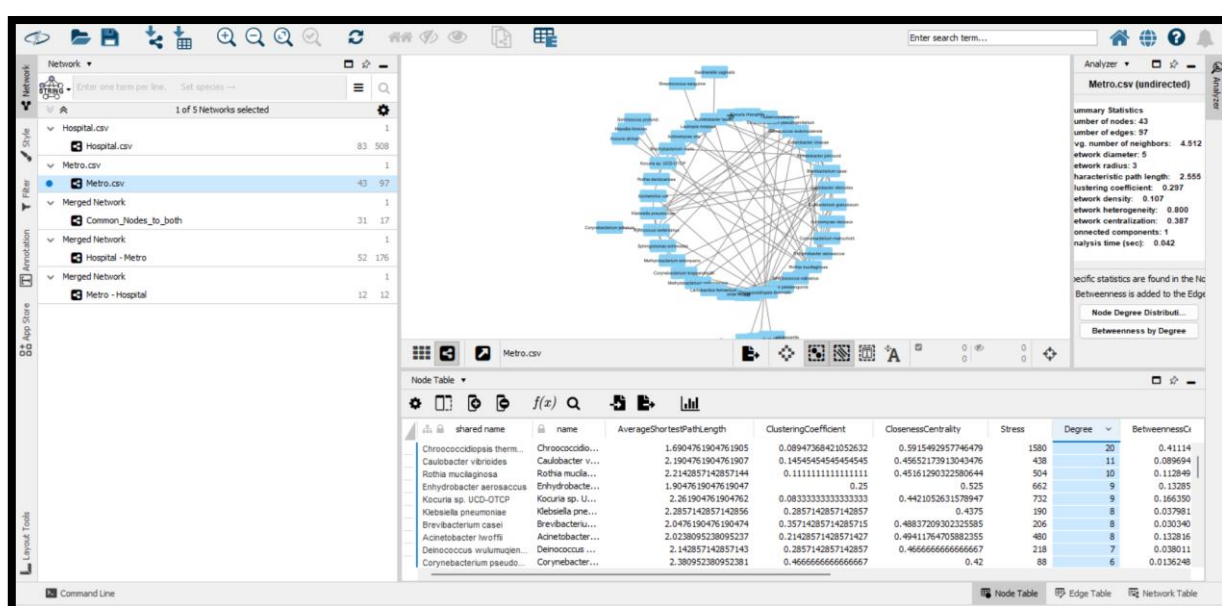


Figure 3.4.5: Top interacting microbes in the Metro Network

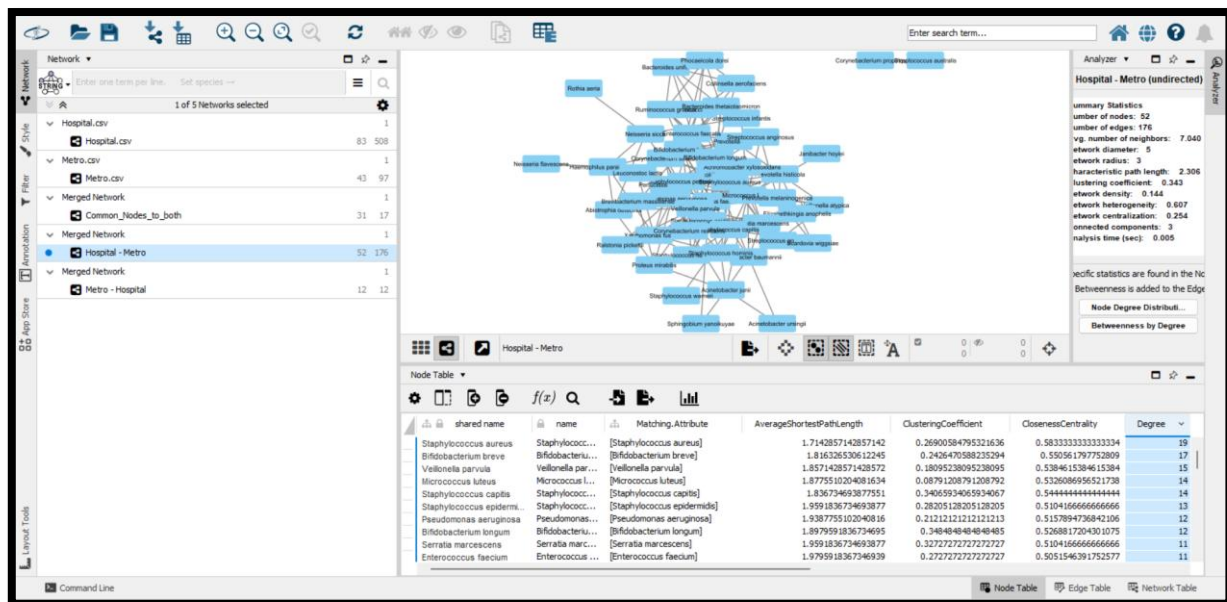


Figure 3.4.6: Top interacting microbes in the Unique Hospital Network

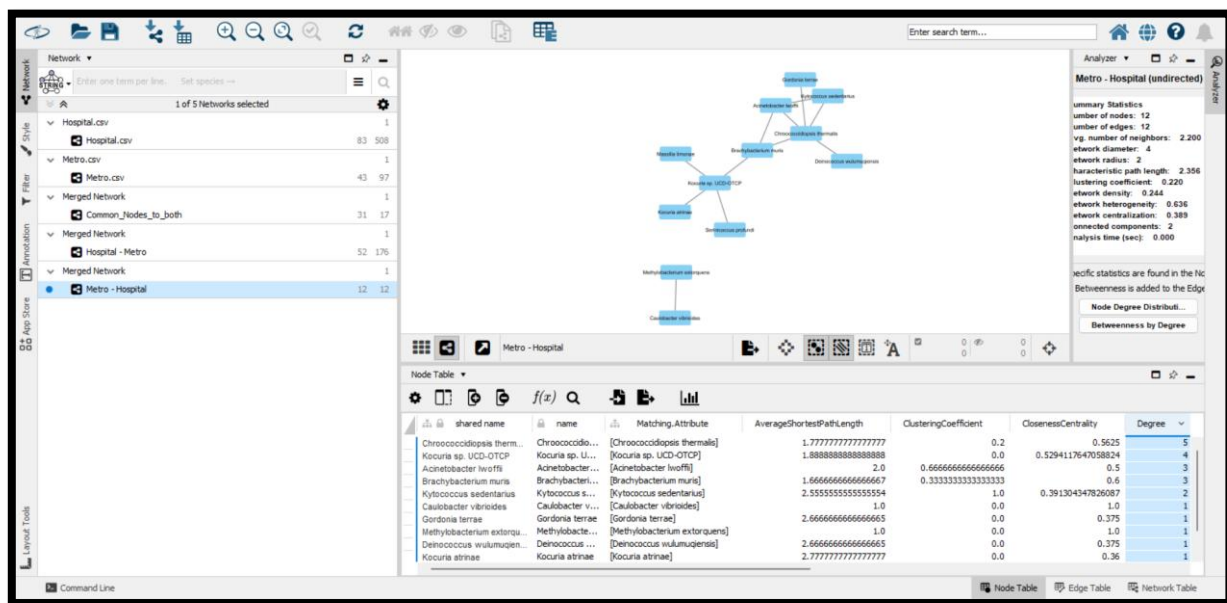


Figure 3.4.7: Top interacting microbes in the Unique Metro Network

In case of the Hospital Network *Staphylococcus aureus*, had the most number of interactions both before and after the removal of common nodes. Additionally it's degree has not gone down by much after removing common interactions, indicate potentially unique interactions of these microbes in the Hospital environment.

In contrast, in the Metro Network though *Chroococcidiopsis thermalis* has the highest number of interactions both before and after the removal of common nodes, its degree takes a sharp hit falling from 20 to 5. This potentially indicates that most of its interactions are not unique to the Metro environment unlike in the first case.

Additionally, it is also noted that in the case of the Metro Network, its unique interaction forms a small and sparse graph, which indicates there were probably not many environment specific interactions to begin with.

Discussions:

Regarding Question 3 part 2, I was unaware that average clustering coefficient was a measure and thought it was a typo, therefore I asked Krushanu, Sidharth and the TA regarding the same.
